1. Intro

This documentation describes the architecture and structure of the Taxi Livo web application. The purpose of this document is to provide insight into the application's technical structure, the architecture used, and the way the different components of the application interact with one another.

In addition, this documentation serves as a reference for developers, administrators, and other stakeholders who need to understand, maintain, or further develop the application. It covers the overall architecture, the technologies used, the project structure, and the key components that make up the web application.

By clearly documenting the application's structure and design decisions, it becomes easier to add new functionality, implement changes, and ensure the quality, maintainability, and long-term sustainability of the Taxi Livo web application.

2. Project Structure

The Taxi Livo web application is built using React as its frontend framework. The application uses JavaScript (JSX) for its logic, HTML (through JSX) for its structure, and CSS for its styling. The backend is powered by Supabase, which is responsible for the database, authentication, and API functionality. The application is built and bundled using Vite, providing fast development and efficient build performance.

The project is organized according to the following structure:

```
taxi-app-1.0/
│
├── .vscode/
├── dist/
├── docs/
├── node_modules/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   ├── utils/
│   ├── App.jsx
```

```
│   ├── index.css
│   ├── main.jsx
│   └── supabase.js
│
├── supabase/
├── .env
├── .gitignore
├── deno.lock
├── eslint.config.js
├── generate-icons.js
├── index.html
├── package.json
├── package-lock.json
├── README.md
├── verify-supabase.js
└── vite.config.js
```

Explanation of Each Directory

.vscode

Contains settings specific to Visual Studio Code, such as recommended extensions and workspace configuration. This directory does not affect the functionality of the web application.

dist

Contains the compiled production build of the application. When the project is built using Vite (npm run build), all generated files are placed here. This directory is used for deployment to a web server.

docs

Contains all technical project documentation, including the architecture, installation guides, and application documentation.

node_modules

Contains all external packages and dependencies installed via npm. This directory is generated automatically and should not be modified manually.

public

Contains static files that are served directly to the browser, such as images, favicon files, and other assets that are not processed by React.

The src Directory

The src directory contains the complete source code of the React application.

assets
Contains images, icons, logos, and other static assets used throughout the application.

components
This directory contains all React components that make up the user interface. Organizing the application into components keeps the code modular, reusable, and easy to maintain.

ActionDialog
Responsible for displaying confirmation dialogs or actions that require user confirmation.

Files:
ActionDialog.jsx
ActionDialog.css
Login

Contains the application's login screen. Drivers and other users authenticate through Supabase Authentication.

Files:
Login.jsx
Login.css
MyTripLogs

Displays an overview of a driver's trip records. Users can view detailed trip information from this component.

Files:
MyTripLogs.jsx
MyTripLogs.css
Navigation

Contains the application's navigation, such as the menu or navigation bar used to switch between different screens.

Files:
Navigation.jsx
Navigation.css
RittenstaatForm

One of the core components of the application. It allows users to enter and process trip information before it is stored in Supabase.

Files:
RittenstaatForm.jsx
RittenstaatForm.css
SignatureDialog

Provides functionality for capturing a digital signature when a trip is completed or confirmed.

Files:
SignatureDialog.jsx
SignatureDialog.css
utils

Contains utility functions that are used throughout the application. Centralizing these helper functions prevents code duplication and improves maintainability.

App.jsx
The main component of the React application. It combines the various components and determines which parts of the application are displayed.

main.jsx
The entry point of the React application. It initializes React, mounts the application to the HTML document, and loads App.jsx.

index.css
Contains the global styles that apply to the entire application.

supabase.js
Contains the configuration for connecting to Supabase. It initializes the Supabase client, allowing the application to communicate with the database and authentication services.

Other Files
supabase/

Contains Supabase-related configuration files, as well as any Edge Functions or database migrations associated with the project.

.env
Contains environment variables, such as:

Supabase URL
Supabase API Key

Sensitive information is kept outside the source code by storing it in this file.

.gitignore
Specifies which files and directories should not be tracked by Git, including:

node_modules
.env
dist
eslint.config.js

Configuration file for ESLint. This tool checks the JavaScript code for potential errors and enforces a consistent coding style.

generate-icons.js
A script used to automatically generate or process application icons.

index.html
The HTML page that loads the React application. React then takes control of rendering the application's content.

package.json
Contains the project's metadata, dependencies, and available scripts, including:

npm install
npm run dev
npm run build
package-lock.json

Locks the exact versions of all installed npm packages, ensuring that every developer uses the same dependency versions.

README.md
Contains general project documentation, including installation instructions and information for developers.

verify-supabase.js
A script used to verify that the connection to Supabase is functioning correctly.

vite.config.js
The configuration file for Vite. It manages settings for the development server, build process, and any plugins used by the application.

2. Authentication

This chapter briefly describes how the login and password recovery functionality is implemented using Supabase. It serves as a reference for future developers.

Supabase Configuration (`src/supabase.js`)

The Supabase client is initialized using `createClient` from `@supabase/supabase-js`. The required credentials (`VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY`) are loaded from the `.env` environment variables.

User Login (`src/App.jsx` & `src/components/Login.jsx`)

- **UI Component:** `Login.jsx` contains all authentication-related forms (login, forgot password, and password reset) and manages its own state to determine which view is displayed.
- **Authentication Logic:** The actual login functionality is implemented in `App.jsx` through the `handleLogin` function, which uses the Supabase SDK:

  supabase.auth.signInWithPassword({ email, password })

- **Session Management:** `App.jsx` listens for authentication events using `supabase.auth.onAuthStateChange`. After a successful login, the active session is detected, the `isLoggedIn` state is set to `true`, and the session state is stored in `localStorage` (`taxilivo_auth`) to persist authentication across page reloads.

Forgot Password (Requesting a Reset Link) (`src/components/Login.jsx`)

- In the **Forgot Password** view, the user enters their email address.
- The `handleForgotPasswordRequest` function in `Login.jsx` calls:

  supabase.auth.resetPasswordForEmail(email)

- Supabase handles sending the secure password reset email based on the authentication settings configured in the Supabase project dashboard.

Password Reset (`src/components/Login.jsx` & `src/App.jsx`)

- **Detecting the Reset Link:** When the user clicks the password reset link in the email and returns to the application, `App.jsx` detects the recovery flow either through the URL recovery token or the `PASSWORD_RECOVERY` authentication event emitted by Supabase.
- The `isRecovering` state is set to `true`, causing `Login.jsx` to immediately display the `update_password` view.
- **Saving the New Password:** The `handlePasswordUpdateSubmit` function in `Login.jsx` updates the user's password by calling:

  supabase.auth.updateUser({ password: newPassword })

- **Completion:** After the password has been successfully updated, the `handlePasswordUpdatedComplete` callback in `App.jsx` is executed. As a security measure, the user is immediately signed out using `supabase.auth.signOut()`, after which they are required to log in again using their new password.

Error Handling

Specific Supabase error messages (such as rate limits, invalid login credentials, or network errors) are caught in `Login.jsx` using `try...catch` blocks. These errors are translated into clear, user-friendly Dutch error messages before being displayed to the user.

3. Trip Logs Architecture and Data Flow

This document describes the architecture and data flow of the Trip Logs (including Delivery Notes) from data entry on the home page through storage and display on the **My Trip Logs** page.

Home Page (`src/components/RittenstaatForm.jsx`)

The home page is implemented as the `RittenstaatForm` component. This is where drivers enter their trip logs and delivery notes.

- **State Management:** The form manages several local state variables:
  - `headerLeft` & `headerRight`: General information such as vehicle registration, date, driver name, and odometer readings.
  - `trips`: An array containing the standard 17 trip entries.
  - `pakbonnen`: A dynamic array of delivery notes. Drivers can add new delivery notes using `addPakbon` and remove existing ones.
  - `signature`: The driver's signature, stored as a Data URL and captured through a separate `SignatureDialog` component.
- **Auto-Save (Draft):** Whenever a form field or delivery note is modified, a `useEffect` hook triggers the `onFormChange` function. This callback, provided by `App.jsx`, immediately stores the current form data in `localStorage` under the key `taxilivo_draft`. This ensures that drivers do not lose their work if the application is closed unexpectedly.

Saving and Synchronization (`src/App.jsx`)

The **Save** and **Submit** actions initiated from `RittenstaatForm` are handled by `App.jsx`, which acts as the bridge between the user interface and the Supabase backend.

- **Local Storage:** When a trip log is saved (`handleSaveLog`), it is first added to the local React state (`logs`). This provides immediate feedback to the user (e.g., *"Trip log saved locally"*).
- **Cloud Synchronization:** After the local save, the application synchronizes the data with Supabase in the background:
  - The parent form object is converted using `mapStateToDbForm` and written to the `forms` table using `.upsert()`.
  - Associated delivery notes are linked to the corresponding `form_id` and stored in the `pakbonnen` table using `.upsert()`.

    - Delivery notes that have been removed locally are automatically deleted from Supabase using `.delete()` to keep the database synchronized.
- **Submitting a Trip Log:** When the user clicks **Submit**, or when `handleSendLog` is called, the application first generates a PDF and sends it via the Supabase Edge Function `send-email`. If the operation succeeds, the trip log status is updated to `'submitted'`, the corresponding record in Supabase is updated, and the local draft is removed from storage.

My Trip Logs (`src/components/MyTripLogs.jsx`)

This component is responsible for displaying the driver's trip log history. The data (`logs`) is retrieved by `App.jsx` from Supabase when the user logs in, using a relational query that loads records from the `forms` table together with their associated `pakbonnen`.

- **Display and Sorting:** Trip logs are sorted chronologically (`sortedLogs`) and displayed as individual log cards.
- **Information and Status:** Each card displays key information, including:
    - Date
    - Driver
    - Vehicle
    - Status badge (*Saved* or *Submitted*)
    - Delivery note indicator (e.g., **Delivery Notes (2)**)
- **Available Actions for Each Trip Log:**
    - **Open/Edit:** Reloads the selected trip log into `RittenstaatForm` as a draft, provided its status is still `saved`.
    - **Submit:** Users can submit a previously saved trip log directly from this overview using the **Submit** button (`onSubmit`). This sends the email and updates the status to `submitted`.
    - **Delete:** Removes the trip log using `handleDeleteLog`. The log is removed immediately from the local interface and then permanently deleted from Supabase in the background.
- **Bulk Actions:** Users can select multiple trip logs using checkboxes and perform batch operations, such as downloading them as a single merged PDF or deleting multiple logs simultaneously using `handleDeleteLogs`.

4. Trip Log Submission Flow (Supabase)

This section describes the workflow that is executed when a driver chooses to permanently submit a trip log (with or without associated delivery notes). The architecture is designed to perform server-side operations, such as sending emails, securely through **Supabase Edge Functions**.

What Happens When the User Clicks Submit?

The submission logic is primarily implemented in the `submitTripLog()` function and its related handlers in `src/App.jsx`.

1. **Local PDF Generation**

   Before communicating with the server, the application uses `generatePdfBlob(logData)` (from `src/utils/pdfGenerator.js`) to generate a binary PDF Blob containing the trip log and any associated delivery notes directly in the browser.

2. **Building the Payload**

   To minimize overhead (such as Base64 encoding), the generated PDF is combined with metadata—including the vehicle number, driver name, and date—into a native `FormData` object.

3. **Invoking the Supabase Edge Function**

   The application calls:

   supabase.functions.invoke('send-email', { body: formData })

   This invokes the secure `send-email` Edge Function hosted on the Supabase server.

4. **Server-Side Processing with Resend**

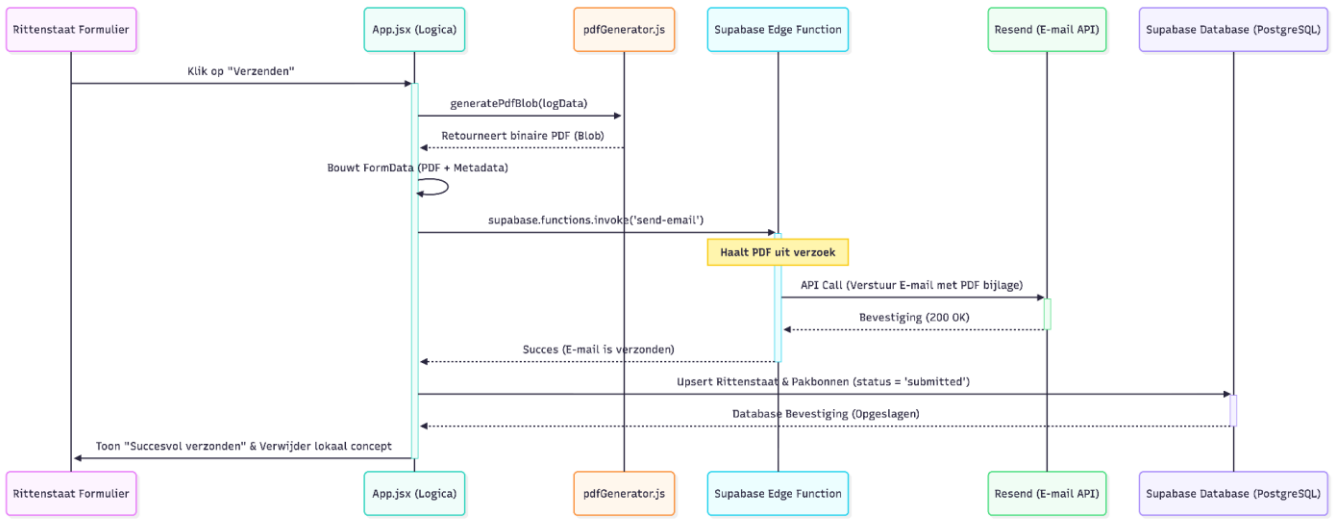
   The Edge Function extracts the PDF from the `FormData` object and sends it using **Resend**, which is responsible for the reliable delivery of the email and its PDF attachment to the central office.

5. **Database Synchronization**

   Only after the email has been sent successfully is the trip log status updated to `'submitted'` in the React application. The application then performs an `.upsert()` to the Supabase database, permanently storing the trip log with `submitted_at = now()`. Finally, the local draft cache (`taxilivo_draft` in `localStorage`) is removed to complete the submission process.

Visual Representation (Data Flow)

The following Mermaid sequence diagram illustrates the flow of data during the submission process.

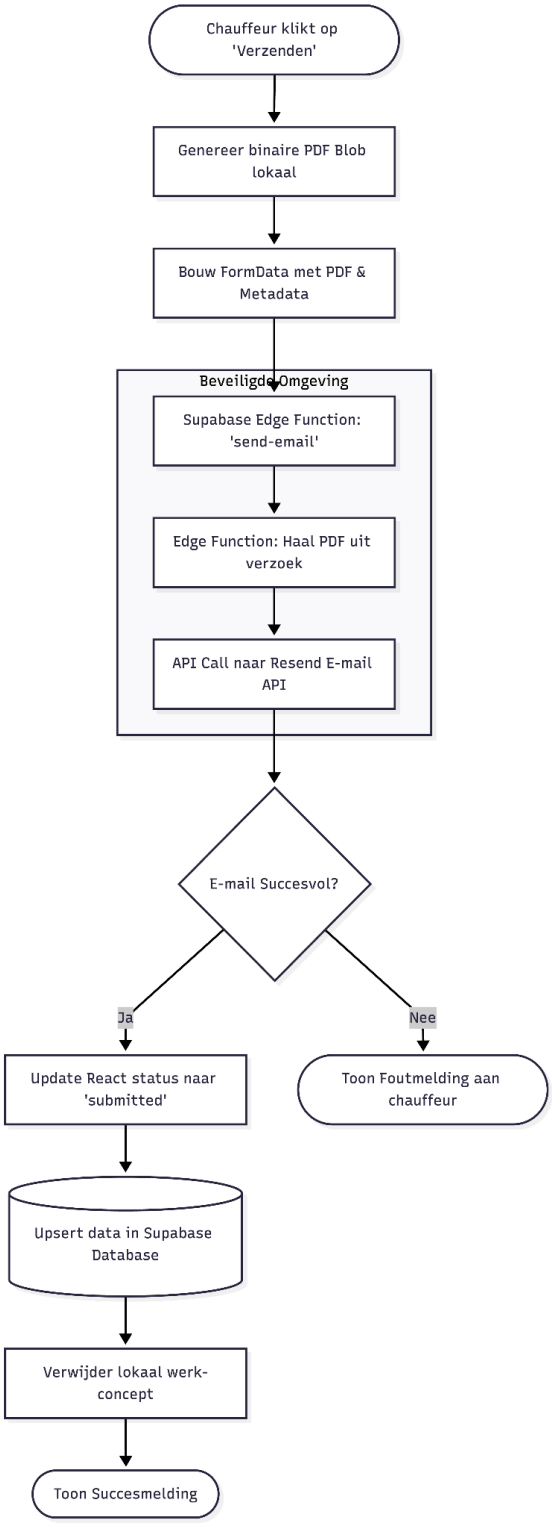


```mermaid
sequenceDiagram
    participant UI as Rittenstaat Formulier
    participant App as App.jsx (Logica)
    participant PDF as pdfGenerator.js
    participant Edge as Supabase Edge Function
    participant Resend as Resend (E-mail API)
    participant DB as Supabase Database (PostgreSQL)

    UI->>App: Klik op "Verzenden"
    activate App
    App->>PDF: generatePdfBlob(logData)
    PDF-->>App: Retourneert binaire PDF (Blob)
```

```
    App->>App: Bouwt FormData (PDF + Metadata)

    App->>Edge: supabase.functions.invoke('send-email')
    activate Edge
    Note over Edge: Haalt PDF uit verzoek
    Edge->>Resend: API Call (Verstuur E-mail met PDF bijlage)
    activate Resend
    Resend-->>Edge: Bevestiging (200 OK)
    deactivate Resend
    Edge-->>App: Succes (E-mail is verzonden)
    deactivate Edge

    App->>DB: Upsert Rittenstaat & Pakbonnen (status = 'submitted')
    activate DB
    DB-->>App: Database Bevestiging (Opgeslagen)
    deactivate DB

    App->>UI: Toon "Succesvol verzonden" & Verwijder lokaal concept
    deactivate App
```

3. Flowchart

In addition to the interaction diagram above, the following flowchart illustrates the logical sequence of the submission process, including error handling.



```mermaid
flowchart TD
    Start([Chauffeur klikt op 'Verzenden']) --> GenPDF[Genereer binaire PDF Blob lokaal]
    GenPDF --> FormData[Bouw FormData met PDF & Metadata]
    FormData --> EdgeCall[Supabase Edge Function: 'send-email']

    subgraph Server-Side [Beveiligde Omgeving]
        EdgeCall --> Extract[Edge Function: Haal PDF uit verzoek]
        Extract --> Resend[API Call naar Resend E-mail API]
    end

    Resend --> SuccessCheck{E-mail Succesvol?}

    SuccessCheck -- Ja --> UpdateStatus[Update React status naar 'submitted']
    UpdateStatus --> UpsertDB[(Upsert data in Supabase Database)]
    UpsertDB --> ClearDraft[Verwijder lokaal werk-concept]
    ClearDraft --> EndSuccess([Toon Succesmelding])

    SuccessCheck -- Nee --> EndError([Toon Foutmelding aan chauffeur])
```

5. Database Architectuur & Connectie Overzicht

This document provides a high-level overview of the interaction between the React frontend (Web Application), the Supabase platform (API & Authentication), and the underlying PostgreSQL database. Its purpose is to give future developers a clear understanding of the complete data flow, from user authentication to storing and permanently submitting a trip log.

The Three Layers of the Architecture

Frontend (React Web Application)

The frontend runs in the user's browser and provides the application's user interface. User interactions take place here, application state is managed locally (using React state and `localStorage`), and PDF documents are generated before submission.

Supabase (Backend-as-a-Service)

Supabase acts as the middleware between the frontend and the database. It provides ready-to-use services, including:

- **Supabase Auth** for user authentication.
- **PostgREST**, a REST API for interacting with the PostgreSQL database.
- **Edge Functions**, which allow secure execution of server-side code.

PostgreSQL (Database)

PostgreSQL is the underlying relational database that powers Supabase. It stores application data in tables such as `forms` (trip logs) and `pakbonnen` (delivery notes), while user authentication data is securely managed in Supabase's protected `auth.users` table.

Data Flow (User Journey)

User Login

When a user opens the application and signs in, the frontend sends the user's email address and password to **Supabase Auth**. Supabase securely validates these credentials against the `auth.users` table in PostgreSQL. If authentication is successful, the frontend receives a session containing a JWT token, which authorizes the user for subsequent requests.

Loading Trip History (My Trip Logs)

After the user has successfully signed in, the application retrieves previously saved trip logs from the database. Using Supabase's relational query capabilities, a single request fetches both the `forms` records and their associated `pakbonnen` records from PostgreSQL.

Creating and Saving a Trip Log

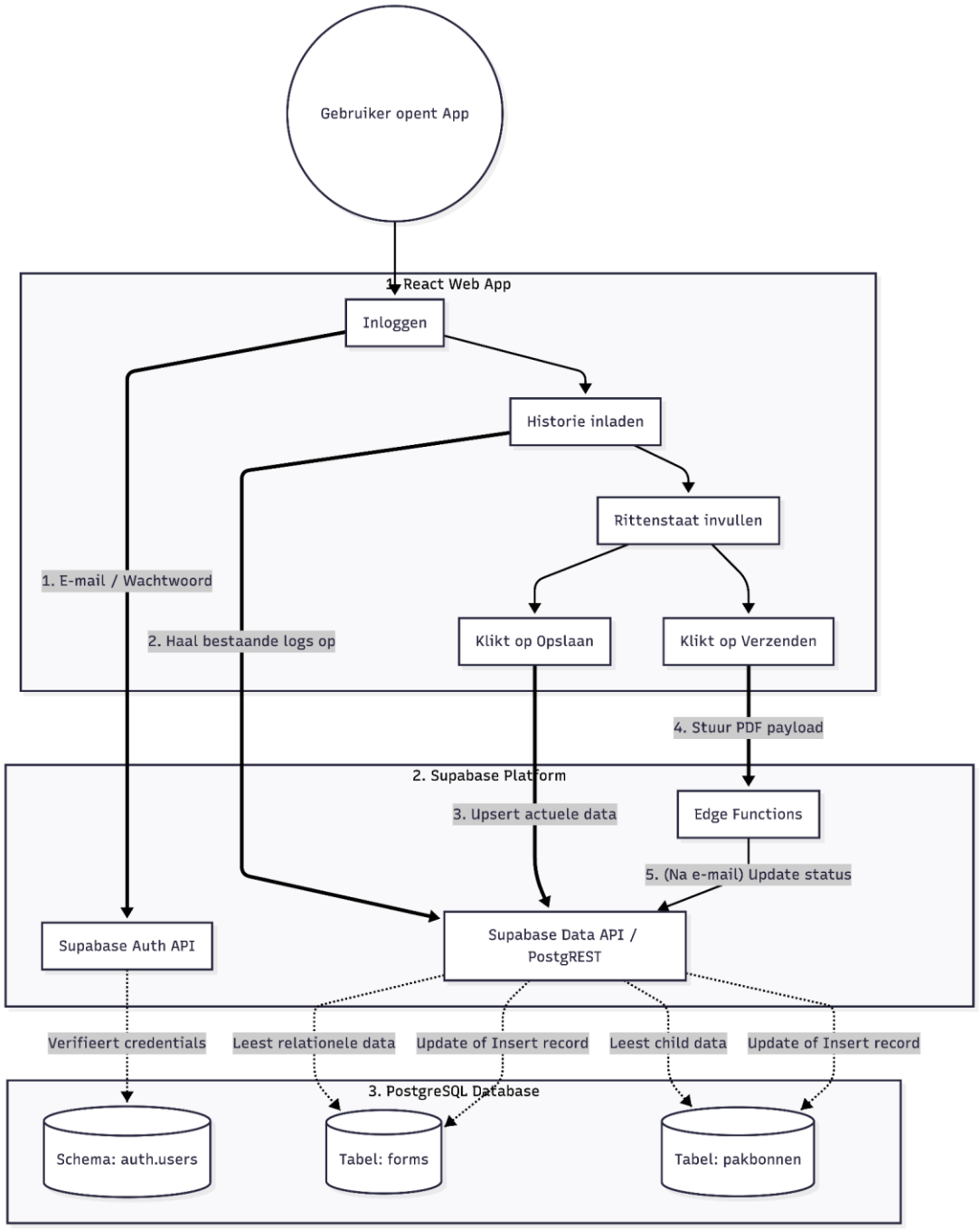
When the user completes a trip log and/or adds delivery notes, clicking **Save** (or triggering the automatic local draft save) stores the data. Database synchronization is performed through the Supabase API using an `.upsert()` operation (update or insert). The data is then written directly to the `public.forms` and `public.pakbonnen` tables in PostgreSQL.

Submitting a Trip Log

Once the trip log is complete and the user clicks **Submit**, the process follows a secure server-side workflow to enable email delivery. The application first generates a PDF locally in the browser and sends it to a **Supabase Edge Function**. After the Edge Function successfully delivers the email using **Resend**, the application performs one final update through the Supabase API, marking the corresponding database record in PostgreSQL with the status `'submitted'`.

Flowchart

The diagram below illustrates the complete lifecycle of the application's data, clearly showing the separation between the **Supabase platform** (its APIs and backend services) and the underlying **PostgreSQL database**.




```mermaid
flowchart TD
```

```
%% Gebruiker / Start
Start((Gebruiker opent App)) --> Login

%% Frontend Laag
subgraph Frontend [1. React Web App]
    Login[Inloggen]
    Overzicht[Historie inladen]
    VulIn[Rittenstaat invullen]
    Opslaan[Klikt op Opslaan]
    Verzenden[Klikt op Verzenden]

    Login --> Overzicht
    Overzicht --> VulIn
    VulIn --> Opslaan
    VulIn --> Verzenden
end

%% Supabase API Laag
subgraph Supabase [2. Supabase Platform]
    AuthAPI[Supabase Auth API]
    RestAPI[Supabase Data API / PostgREST]
    Edge[Edge Functions]
end

%% PostgreSQL Database Laag
subgraph Database [3. PostgreSQL Database]
    DB_Users[(Schema: auth.users)]
    DB_Forms[(Tabel: forms)]
    DB_Pakbonnen[(Tabel: pakbonnen)]
end

%% Connecties Inloggen
Login == "1. E-mail / Wachtwoord" ==> AuthAPI
AuthAPI -. "Verifieert credentials" .-> DB_Users

%% Connecties Historie Inladen
Overzicht == "2. Haal bestaande logs op" ==> RestAPI
RestAPI -. "Leest relationele data" .-> DB_Forms
RestAPI -. "Leest child data" .-> DB_Pakbonnen

%% Connecties Opslaan
Opslaan == "3. Upsert actuele data" ==> RestAPI
RestAPI -. "Update of Insert record" .-> DB_Forms
RestAPI -. "Update of Insert record" .-> DB_Pakbonnen
```

```
    %% Connecties Verzenden
    Verzenden == "4. Stuur PDF payload" ==> Edge
    Edge -- "5. (Na e-mail) Update status" --> RestAPI
```

6. Database

This chapter describes the database schema of the Taxi Livo web application. The application uses a PostgreSQL database managed by Supabase. The database is designed to store trip logs, delivery notes, and user information in a structured, reliable, and secure manner.

The schema consists of three main tables: `auth.users`, `forms`, and `pakbonnen`. The `auth.users` table contains information about authenticated users, while the `forms` table stores trip logs and the `pakbonnen` table stores the associated delivery note records.

These tables are linked through primary and foreign key relationships, ensuring data integrity and maintaining the relationships between users, trip logs, and their corresponding delivery notes.

auth.users			
uuid	id	PK	Primary Key
string	email		User Email

forms			
uuid	id	PK	Primary Key
uuid	user_id	FK	References auth.users(id)
timestamp	created_at		Created Timestamp
timestamp	updated_at		Updated Timestamp
timestamp	submitted_at		Submission Timestamp
text	status		saved | submitted
integer	km_stand_begin		Start Odometer
integer	km_stand_eind		End Odometer
numeric	liters_getankt		Fuel Liters
text	ruimte_kantoor		Office Notes
text	wagen_nummer		Vehicle Number
text	kenteken		License Plate
text	naam		Driver Name
date	datum		Shift Date
text	dag		Day of Week
time	werktijd_van		Shift Start Time
time	werktijd_tot		Shift End Time
text	signature		Driver Signature Data
jsonb	rides		JSONB Array of Rides

pakbonnen			
uuid	id	PK	Primary Key
uuid	form_id	FK	References forms(id) ON DELETE CASCADE
uuid	user_id	FK	References auth.users(id) ON DELETE CASCADE
text	chauffeur		Chauffeur Name
text	kenteken		Vehicle License Plate
text	scheepsnaam		Ship/Vessel Name
text	bedrijf		Client Company Name
text	naam		Customer Recipient Name
text	signature		Customer Signature Data
text	van		Pickup Location
text	naar		Destination Location
date	datum		Delivery Date
time	tijd		Delivery Time
numeric	ritprijs		Trip Price
timestamp	created_at		Created Timestamp
timestamp	updated_at		Updated Timestamp

```
erDiagram
    "auth.users" {
        uuid id PK "Primary Key"
        string email "User Email"
    }
    forms {
        uuid id PK "Primary Key"
        uuid user_id FK "References auth.users(id)"
        timestamp created_at "Created Timestamp"
        timestamp updated_at "Updated Timestamp"
```

```
    timestamp submitted_at "Submission Timestamp"
    text status "saved | submitted"
    integer km_stand_begin "Start Odometer"
    integer km_stand_eind "End Odometer"
    numeric liters_getankt "Fuel Liters"
    text ruimte_kantoor "Office Notes"
    text wagen_nummer "Vehicle Number"
    text kenteken "License Plate"
    text naam "Driver Name"
    date datum "Shift Date"
    text dag "Day of Week"
    time werktijd_van "Shift Start Time"
    time werktijd_tot "Shift End Time"
    text signature "Driver Signature Data"
    jsonb rides "JSONB Array of Rides"
}
pakbonnen {
    uuid id PK "Primary Key"
    uuid form_id FK "References forms(id) ON DELETE CASCADE"
    uuid user_id FK "References auth.users(id) ON DELETE CASCADE"
    text chauffeur "Chauffeur Name"
    text kenteken "Vehicle License Plate"
    text scheepsnaam "Ship/Vessel Name"
    text bedrijf "Client Company Name"
    text naam "Customer Recipient Name"
    text signature "Customer Signature Data"
    text van "Pickup Location"
    text naar "Destination Location"
    date datum "Delivery Date"
    time tijd "Delivery Time"
    numeric ritprijs "Trip Price"
    timestamp created_at "Created Timestamp"
    timestamp updated_at "Updated Timestamp"
}
```